Distributed Job Scheduling

on time, once (in effect), even when servers die

Watch the 4-min video

In one line: run either one active scheduler chosen by **leader election with leases**, or a **delayed-job store** where many workers **claim due jobs atomically** with lease timeouts; accept that a job may occasionally run twice and make its effect **idempotent**; and decide explicitly how to handle missed runs, time zones and daylight saving.

1 Cron on one server

An app sends a million reminders a day: take your medication at 8, your appointment is tomorrow, your trial ends in three days. One server running cron works until it dies at 7:59, and nobody gets reminded. Add a second server running the same cron, and everyone gets two. Scheduling at scale needs a design for three things: **who fires**, **how work is claimed**, and **what happens on failure**.

2 One active scheduler: leader election

For a modest number of recurring jobs ("rebuild the report at 02:00"), run several scheduler instances but let only the one holding a **lease** fire jobs. The lease is a record with a holder and an expiry in a consistent store: etcd or a Kubernetes `Lease` object, ZooKeeper, Consul, or a database row updated with a conditional write. The leader renews it every few seconds; if it dies, the lease expires and a follower acquires it.

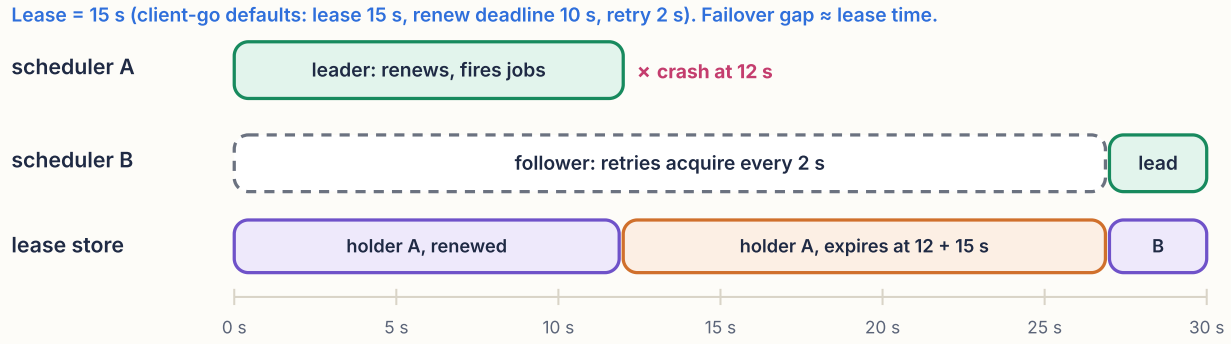


Figure 1. Lease-based leader election. B takes over only after A's lease expires, so there is a gap of up to one lease period with no leader.

Two subtleties. A leader that **pauses** (a long garbage-collection pause, a frozen VM) can wake up after its lease expired and still believe it's the leader, so two schedulers fire briefly. The robust fix is a **fencing token**: each new lease gets a higher number, and the systems the job writes to reject writes with an older token. And the scheduler must track what it has fired durably (a run table), so the new leader knows whether the 02:00 job ran.

Off-the-shelf: Kubernetes **CronJob** (the controller is already leader-elected), Quartz in clustered mode (database row locks), or managed schedulers (EventBridge Scheduler, Cloud Scheduler, Azure Logic Apps / Functions timers). Note that the Kubernetes docs themselves warn a CronJob may create two Jobs, or none, for one schedule in rare cases; it recommends idempotent jobs, and `concurrencyPolicy: Forbid` plus `startingDeadlineSeconds` control overlap and lateness.

3 A million jobs: the delayed-job store

A million individual reminders don't belong in a crontab; they are **data**. Each job is a row with a `due_at` time and a status, in a database table or a Redis **sorted set** scored by timestamp (`ZADD reminders <epoch> <id>`, then `ZRANGEBYSCORE reminders -inf <now>` to find due ones). Workers poll for due jobs every few seconds, claim them, and run them. Recurring schedules are stored as a rule (cron expression or an RFC 5545 RRULE) plus the user's time zone; after each run the next occurrence is computed and inserted. For very high timer volumes in memory, systems such as Kafka and Netty use **hierarchical timing wheels**.

4 Claiming without duplicates

Claiming is where duplicates come from: two workers read the same due row, and both send. The claim must be **atomic**:

- A conditional update: `UPDATE … SET status='running' WHERE id=? AND status='pending'`, and only the worker whose update changed a row owns the job.
- Batch claiming in PostgreSQL or MySQL 8 with `SELECT … FOR UPDATE SKIP LOCKED`: competing workers skip rows another worker has locked, instead of waiting or double-claiming.
- In Redis, a Lua script that pops due members and moves them to an in-flight set in one step (or `ZREM`: whoever removes the member wins).

Every claim carries a **lease** (`lease_until`). If the worker crashes, the lease passes and another worker reclaims the job. Leases should be comfortably longer than the job's normal duration, or a slow worker will be duplicated by a reclaim.

```
-- claim a batch atomically; competing workers skip locked rows
UPDATE reminders SET status = 'running', lease_until = now() + interval '2 minutes',
                     attempts = attempts + 1
WHERE id IN (
  SELECT id FROM reminders
  WHERE (status = 'pending' AND due_at <= now())
     OR (status = 'running' AND lease_until < now())    -- reclaim abandoned jobs
  ORDER BY due_at LIMIT 500
  FOR UPDATE SKIP LOCKED )
RETURNING id, user_id, due_at, payload;

-- index: CREATE INDEX ON reminders (due_at) WHERE status IN ('pending','running');
```

Figure 2. One statement claims up to 500 due or abandoned jobs. The partial index keeps the scan cheap even with millions of finished rows.

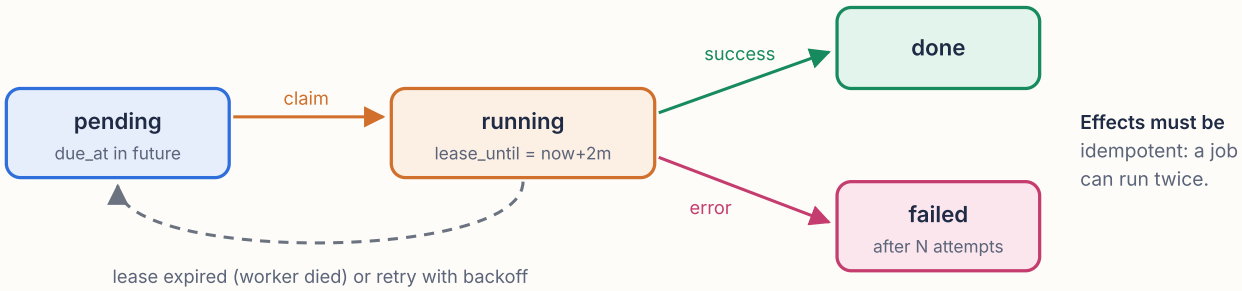


Figure 3. The job lifecycle. The dashed path is the at-least-once part: a job whose worker vanished goes back to be claimed again.

5 At-least-once + idempotent

A job can still run twice: a worker sends the reminder, then crashes before marking it done; the lease expires and another worker sends it again. **Exactly-once firing** across a crash would need the side effect and the status change in one transaction, which is impossible when the side effect is an external call. What is achievable is **at-least-once execution with idempotent effects**: pass an **idempotency key** such as `reminder_id + due_at` to the notification service, which records keys it has seen (with a TTL) and drops repeats. Including `due_at` keeps tomorrow's run of the same recurring reminder distinct.

6 Missed runs, load spikes and time zones

- **Misfire policy**, per job type, after an outage: *catch up* (run every missed occurrence; right for billing), *run once* (right for a report), or *skip if too late* (right for "take your 8:00 pill" at 13:00). Quartz calls these misfire instructions.
- **Thundering herds**: a million reminders a day average about 12 per second, but if a third are set for 8:00 local time, each time zone's 8:00 brings a burst. Pre-claim slightly early, scale workers ahead of known peaks, and add a few seconds of jitter where exact timing doesn't matter.
- **Time zones**: store the user's IANA zone (`Europe/London`), not a fixed UTC offset, because offsets change with daylight saving and with law. Compute the next run in local time, convert it to UTC for `due_at`, and recompute after each run.

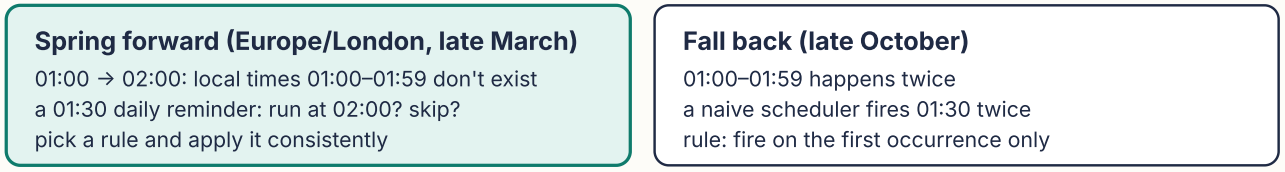


Figure 4. Daylight-saving edge cases. Both need an explicit rule, and a test.

WORKED EXAMPLE: A MILLION SCHEDULED REMINDERS PER DAY

Reminders are rows in a PostgreSQL table with `due_at` (UTC), `status`, `lease_until`, `attempts`, and a partial index on due time for unfinished rows. **Twenty workers** poll every few seconds, each claiming up to 500 due reminders with `SKIP LOCKED` and a **two-minute lease** (the query above). Each send calls the notification service with `reminder_id:due_at` as the **idempotency key**; then the row is marked done, and for recurring reminders the next occurrence is computed in the user's zone and inserted. Twenty workers × 500 per claim every few seconds gives thousands per second of headroom over the 12/s average, enough for the 8:00 bursts. After an outage, overdue reminders are sent if they're **less than an hour late** and marked skipped otherwise, with a metric for each, so a medication reminder never arrives at dinner time.

COMMON MISTAKES

- The same cron on every replica: every job fires N times.
- Read-then-update claiming without a lock or condition, so two workers take one job.
- Storing a UTC offset instead of an IANA time zone, breaking at the next DST change.
- No misfire policy: after an outage, a flood of stale jobs fires at once.

7 Key terms

Leader election: choosing one active instance via a renewable lease

Delayed-job store: jobs as rows or sorted-set members keyed by due time

Misfire policy: what to do with runs missed during an outage

Fencing token: increasing number that lets systems reject a stale leader

SKIP LOCKED: row-lock option letting workers claim different rows concurrently

Idempotency key: identifier that makes a repeated effect a no-op

8 Interview questions

Q1. How do you make sure a scheduled job runs once? BASIC

Use one active scheduler through leader election with leases, or a delayed-job store where workers claim due jobs atomically with lease timeouts, and make the job's effect idempotent so a rare duplicate run is harmless.

Q2. How do you handle time zones? BASIC

Store the user's IANA time zone, compute the next run in local time and store it as UTC, and handle daylight-saving gaps and overlaps with explicit rules.

Q3. Why not run cron on every replica? BASIC

Every replica fires every job, so effects multiply; only a leader should fire, or jobs must be claimed atomically.

Q4. Workers keep sending some reminders twice. What do you check? SCENARIO

Whether claims are atomic, whether leases are shorter than job duration so slow jobs get reclaimed, and whether the send uses an idempotency key.

Q5. The scheduler was down 3 hours. What runs when it returns? SCENARIO

Depends on each job type's misfire policy: billing catches up every run, reports run once, and time-sensitive reminders older than a threshold are skipped and recorded.

Q6. Why isn't leader election alone enough to prevent double execution? TRICKY

A paused leader can resume after its lease expired while a new leader acts; use fencing tokens and idempotent effects.

Q7. Why is exactly-once firing so hard? TRICKY

The external side effect and recording that it happened can't be one atomic step, so a crash between them forces either a possible repeat or a possible loss; at-least-once with idempotency is the practical answer.

Next: SD-V-023 · Green Software backendroadmap.com